In [17]:

```python
import h5py
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
class MNISTDataset(Dataset):
    def __init__(self, file_path):
        with h5py.File(file_path, "r") as f:
            self.x = f["xdata"][:]
            self.y = f["ydata"][:]


        self.x = self.x.astype(np.float32)


        if self.x.max() > 1:
            self.x = self.x / 255.0


        if len(self.y.shape) > 1:
            self.y = np.argmax(self.y, axis=1)

        self.y = self.y.astype(np.int64)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        x = torch.tensor(self.x[idx], dtype=torch.float32)
        y = torch.tensor(self.y[idx], dtype=torch.long)
        return x, y

train_dataset = MNISTDataset("mnist_traindata.hdf5")
test_dataset = MNISTDataset("mnist_testdata.hdf5")

train_loader = DataLoader(train_dataset, batch_size=100, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=100, shuffle=False)

print("train x shape:", train_dataset.x.shape)
print("train y shape:", train_dataset.y.shape)
print("first y sample:", train_dataset.y[0])

x_batch, y_batch = next(iter(train_loader))
print("x_batch shape:", x_batch.shape)
print("y_batch shape:", y_batch.shape)
print("first 10 labels:", y_batch[:10])
print("y_batch dtype:", y_batch.dtype)

model = nn.Sequential(
    nn.Linear(28 * 28, 10)
)
criterion = nn.CrossEntropyLoss()

optimizer = torch.optim.SGD(
    model.parameters(),
```

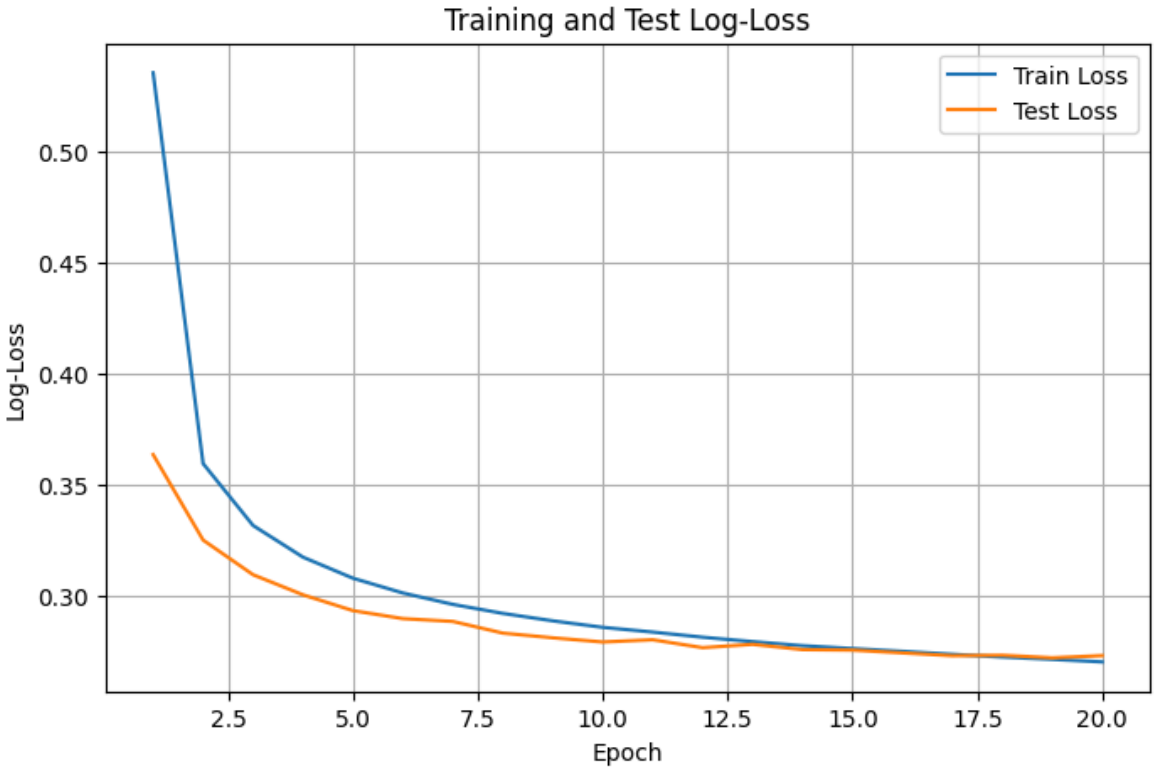
```python
    lr=0.1,
    weight_decay=1e-4

)

def train_one_epoch(model, loader, criterion, optimizer):
    model.train()

    total_loss = 0.0
    correct = 0
    total = 0

    for x, y in loader:
        optimizer.zero_grad()

        outputs = model(x)
        loss = criterion(outputs, y)

        loss.backward()
        optimizer.step()

        total_loss += loss.item() * x.size(0)

        _, predicted = torch.max(outputs, 1)
        correct += (predicted == y).sum().item()
        total += y.size(0)

    avg_loss = total_loss / total
    accuracy = correct / total

    return avg_loss, accuracy


def evaluate(model, loader, criterion):
    model.eval()

    total_loss = 0.0
    correct = 0
    total = 0

    all_preds = []
    all_labels = []

    with torch.no_grad():
        for x, y in loader:
            outputs = model(x)
            loss = criterion(outputs, y)

            total_loss += loss.item() * x.size(0)

            _, predicted = torch.max(outputs, 1)
            correct += (predicted == y).sum().item()
            total += y.size(0)

            all_preds.extend(predicted.cpu().numpy())
            all_labels.extend(y.cpu().numpy())

    avg_loss = total_loss / total
    accuracy = correct / total
```

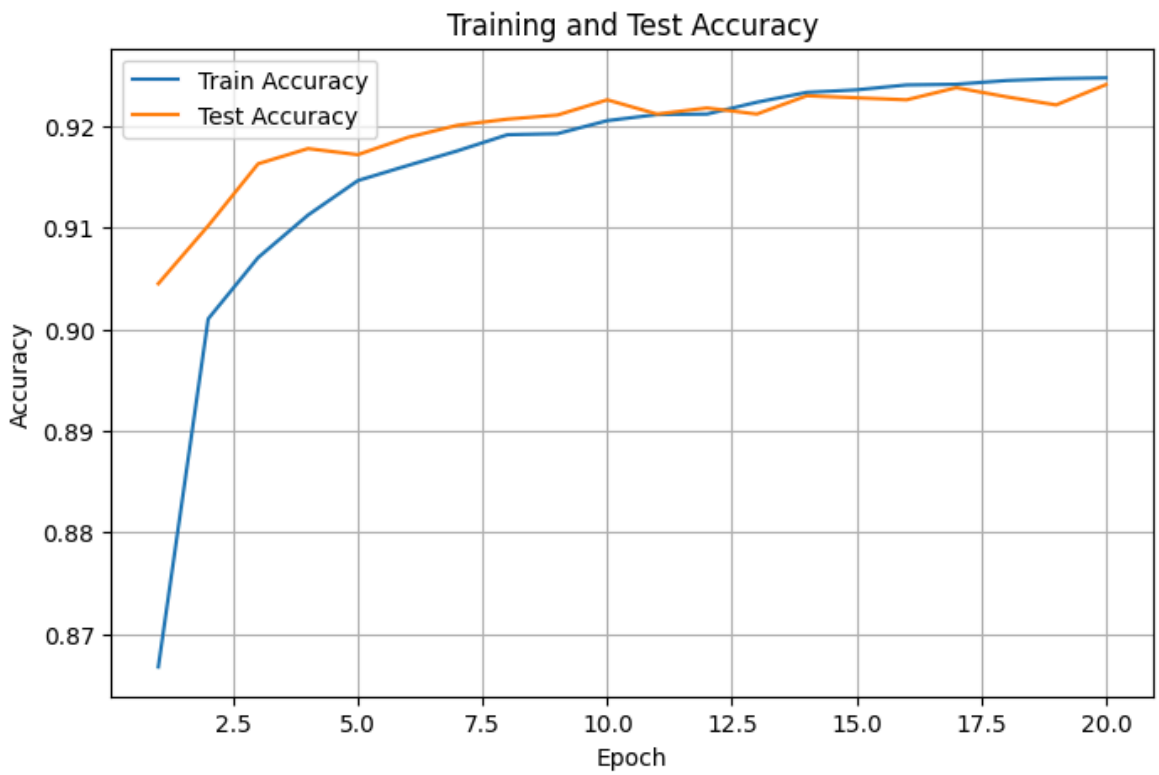
```
    return avg_loss, accuracy, np.array(all_labels), np.array(all_preds)


    num_epochs = 20

train_losses = []
train_accuracies = []
test_losses = []
test_accuracies = []

for epoch in range(num_epochs):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, opti
    test_loss, test_acc, _, _ = evaluate(model, test_loader, criterion)

    train_losses.append(train_loss)
    train_accuracies.append(train_acc)
    test_losses.append(test_loss)
    test_accuracies.append(test_acc)

    print(f"Epoch {epoch+1}/{num_epochs}")
    print(f"Train Loss: {train_loss:.4f}, Train Accuracy: {train_acc:.4f}")
    print(f"Test  Loss: {test_loss:.4f}, Test  Accuracy: {test_acc:.4f}")
    print("-" * 40)

    epochs = range(1, num_epochs + 1)

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(epochs, train_losses, label="Train Loss")
ax.plot(epochs, test_losses, label="Test Loss")
ax.set_xlabel("Epoch")
ax.set_ylabel("Log-Loss")
ax.set_title("Training and Test Log-Loss")
ax.legend()
ax.grid(True)
plt.show()
plt.close(fig)

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(epochs, train_accuracies, label="Train Accuracy")
ax.plot(epochs, test_accuracies, label="Test Accuracy")
ax.set_xlabel("Epoch")
ax.set_ylabel("Accuracy")
ax.set_title("Training and Test Accuracy")
ax.legend()
ax.grid(True)
plt.show()
plt.close(fig)

test_loss, test_acc, y_true, y_pred = evaluate(model, test_loader, criterion)
print("Final Test Accuracy:", test_acc)

num_classes = 10
cm = np.zeros((num_classes, num_classes), dtype=np.int32)

for true_label, pred_label in zip(y_true, y_pred):
    cm[true_label, pred_label] += 1

fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax)
ax.set_xlabel("Predicted Class")
```

```
ax.set_ylabel("True Class")
ax.set_title("Confusion Matrix")
plt.show()
plt.close(fig)
```

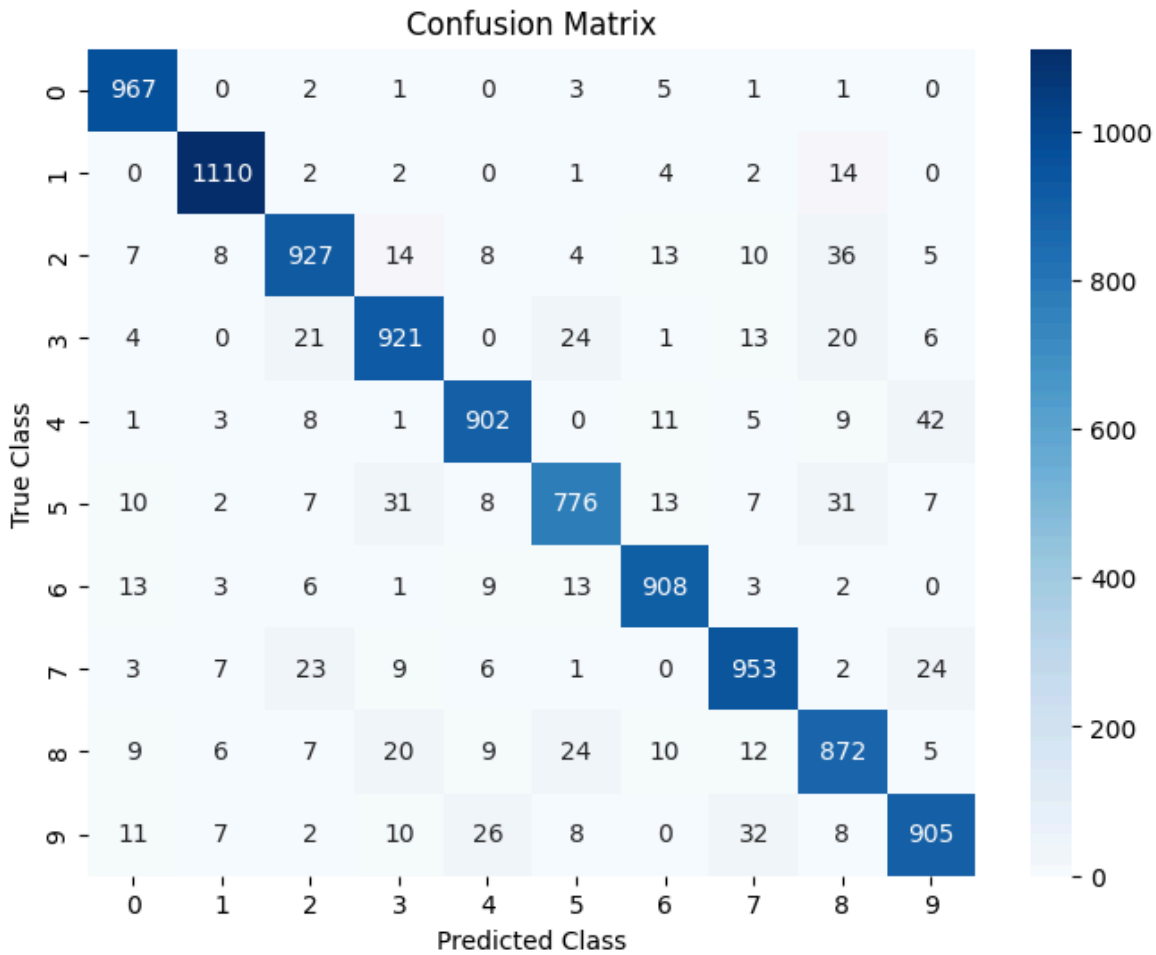
```
train x shape: (60000, 784)
train y shape: (60000,)
first y sample: 5
x_batch shape: torch.Size([100, 784])
y_batch shape: torch.Size([100])
first 10 labels: tensor([6, 6, 9, 5, 5, 9, 0, 5, 3, 9])
y_batch dtype: torch.int64
Epoch 1/20
Train Loss: 0.5357, Train Accuracy: 0.8668
Test  Loss: 0.3635, Test  Accuracy: 0.9045
----------------------------------------
Epoch 2/20
Train Loss: 0.3595, Train Accuracy: 0.9010
Test  Loss: 0.3250, Test  Accuracy: 0.9102
----------------------------------------
Epoch 3/20
Train Loss: 0.3316, Train Accuracy: 0.9071
Test  Loss: 0.3094, Test  Accuracy: 0.9163
----------------------------------------
Epoch 4/20
Train Loss: 0.3173, Train Accuracy: 0.9113
Test  Loss: 0.3003, Test  Accuracy: 0.9178
----------------------------------------
Epoch 5/20
Train Loss: 0.3078, Train Accuracy: 0.9146
Test  Loss: 0.2932, Test  Accuracy: 0.9172
----------------------------------------
Epoch 6/20
Train Loss: 0.3011, Train Accuracy: 0.9161
Test  Loss: 0.2896, Test  Accuracy: 0.9189
----------------------------------------
Epoch 7/20
Train Loss: 0.2960, Train Accuracy: 0.9176
Test  Loss: 0.2883, Test  Accuracy: 0.9201
----------------------------------------
Epoch 8/20
Train Loss: 0.2919, Train Accuracy: 0.9192
Test  Loss: 0.2831, Test  Accuracy: 0.9207
----------------------------------------
Epoch 9/20
Train Loss: 0.2886, Train Accuracy: 0.9193
Test  Loss: 0.2809, Test  Accuracy: 0.9211
----------------------------------------
Epoch 10/20
Train Loss: 0.2857, Train Accuracy: 0.9205
Test  Loss: 0.2790, Test  Accuracy: 0.9226
----------------------------------------
Epoch 11/20
Train Loss: 0.2835, Train Accuracy: 0.9212
Test  Loss: 0.2801, Test  Accuracy: 0.9212
----------------------------------------
Epoch 12/20
Train Loss: 0.2812, Train Accuracy: 0.9212
Test  Loss: 0.2765, Test  Accuracy: 0.9218
----------------------------------------
Epoch 13/20
Train Loss: 0.2792, Train Accuracy: 0.9224
Test  Loss: 0.2780, Test  Accuracy: 0.9212
----------------------------------------
Epoch 14/20
```

```
Train Loss: 0.2774, Train Accuracy: 0.9233
Test  Loss: 0.2756, Test  Accuracy: 0.9230
----------------------------------------
Epoch 15/20
Train Loss: 0.2761, Train Accuracy: 0.9236
Test  Loss: 0.2754, Test  Accuracy: 0.9228
----------------------------------------
Epoch 16/20
Train Loss: 0.2748, Train Accuracy: 0.9241
Test  Loss: 0.2741, Test  Accuracy: 0.9226
----------------------------------------
Epoch 17/20
Train Loss: 0.2735, Train Accuracy: 0.9241
Test  Loss: 0.2729, Test  Accuracy: 0.9238
----------------------------------------
Epoch 18/20
Train Loss: 0.2722, Train Accuracy: 0.9245
Test  Loss: 0.2731, Test  Accuracy: 0.9229
----------------------------------------
Epoch 19/20
Train Loss: 0.2712, Train Accuracy: 0.9247
Test  Loss: 0.2719, Test  Accuracy: 0.9221
----------------------------------------
Epoch 20/20
Train Loss: 0.2701, Train Accuracy: 0.9248
Test  Loss: 0.2729, Test  Accuracy: 0.9241
----------------------------------------
```






```
Final Test Accuracy: 0.9241
```



Model description

A multiclass logistic classifier was implemented in PyTorch for the MNIST dataset. The model was built using nn.Sequential with a single fully connected linear layer that maps the 784-dimensional input vector to 10 output classes. No softmax

activation was added at the output layer because CrossEntropyLoss already includes the softmax operation internally. The model was trained using stochastic gradient descent (SGD) with a mini-batch size of 100. L2 regularization was applied through weight decay to improve generalization.

Training result analysis

The training and test log-loss both decreased steadily over the epochs, while the training and test accuracy both increased and then gradually stabilized. This indicates that the model learned the classification task effectively. The final test accuracy was about 92.4%, which is reasonable for a single-layer logistic classifier on MNIST. The training and test curves stayed close to each other, suggesting that the model did not suffer from severe overfitting.

Confusion matrix analysis

The confusion matrix is reasonable because most values are concentrated along the diagonal, which means the majority of test samples were correctly classified. Some confusion appears between visually similar digits, such as 3 and 5, 4 and 9, and 2 and 8. This is expected because some handwritten digits have similar shapes. Overall, the confusion matrix shows that the classifier learned meaningful decision boundaries for most classes.

In [ ]: